

Double-click (or enter) to edit

```
# 1. Setup: imports & installs (Colab-safe)
!pip -q install wordcloud==1.9.3

import os, zipfile, glob, itertools, io, json, shutil
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from wordcloud import WordCloud

plt.rcParams['figure.figsize'] = (7,4)
print("Imports ready, TensorFlow version:", tf.__version__)

# Optional: reduce TF logging noise
import logging, warnings
warnings.filterwarnings("ignore")
logging.getLogger("tensorflow").setLevel(logging.ERROR)
```

Imports ready, TensorFlow version: 2.19.0 541.6/541.6 kB 9.6 MB/s eta 0:00:00

```
from google.colab import drive
drive.mount('/content/drive')

import os
print(os.listdir("/content/drive/MyDrive"))
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
['Colab Notebooks', 'certificates', 'IMG_20170930_180354.jpg', 'IMG_1789.jpg', 'K.Varun Kumar-Resume.pdf', '2. ISP 10 Seminar pr

```
# === Paths to your two zips on Drive ===
zip_def = "/content/drive/MyDrive/Plastic Bottle Image Dataset.zip" # defective
zip_good = "/content/drive/MyDrive/Water Bottle Dataset.zip" # CHANGE name if different

raw_def_root = "/content/bottle_defects_raw"
raw_good_root = "/content/good_bottles_raw"

import zipfile, shutil

# Clean old folders if they exist
for p in [raw_def_root, raw_good_root]:
    if os.path.exists(p):
        shutil.rmtree(p, ignore_errors=True)

# Extract defective dataset
with zipfile.ZipFile(zip_def, 'r') as z:
    z.extractall(raw_def_root)
print(" Defective extracted to:", raw_def_root)

# Extract good dataset
with zipfile.ZipFile(zip_good, 'r') as z:
    z.extractall(raw_good_root)
print(" Good extracted to:", raw_good_root)

print("Defective top-level:", os.listdir(raw_def_root))
print("Good top-level:", os.listdir(raw_good_root))
```

```
Defective extracted to: /content/bottle_defects_raw
Good extracted to: /content/good_bottles_raw
Defective top-level: ['valid', 'train', 'data.yaml', 'test']
Good top-level: ['Overflowing', 'Half water level', 'Full Water level']
```

```

# Target structure that the CNN will use
base_target = "/content/milk_bottle_defects/milk_bottle_defects"

for split in ["train", "test"]:
    for cls in ["good", "defective"]:
        os.makedirs(os.path.join(base_target, split, cls), exist_ok=True)

print(" Created target structure under:", base_target)

IMAGE_EXTS = (".png", ".jpg", ".jpeg", ".bmp", ".webp", ".jfif")

def list_images(root):
    paths = []
    for r, d, files in os.walk(root):
        for fname in files:
            if fname.lower().endswith(IMAGE_EXTS):
                paths.append(os.path.join(r, fname))
    return paths

# All images from mendemukesh dataset → defective
defect_files = list_images(raw_def_root)

# All images from chethuhn dataset → good
good_files = list_images(raw_good_root)

print("Found GOOD images      :", len(good_files))
print("Found DEFECTIVE images:", len(defect_files))

def split_and_copy(files, cls_name):
    files = sorted(files)
    split = int(0.8 * len(files)) # 80% train, 20% test
    train_f = files[:split]
    test_f = files[split:]

    for src in train_f:
        shutil.copy(src, os.path.join(base_target, "train", cls_name))
    for src in test_f:
        shutil.copy(src, os.path.join(base_target, "test", cls_name))
    print(f"Copied {len(train_f)} train + {len(test_f)} test for class '{cls_name}'")

split_and_copy(good_files, "good")
split_and_copy(defect_files, "defective")

print("Train folders:", os.listdir(os.path.join(base_target, "train")))
print("Test folders :", os.listdir(os.path.join(base_target, "test")))

```

```

Created target structure under: /content/milk_bottle_defects/milk_bottle_defects
Found GOOD images      : 486
Found DEFECTIVE images: 3999
Copied 388 train + 98 test for class 'good'
Copied 3199 train + 800 test for class 'defective'
Train folders: ['defective', 'good']
Test folders : ['defective', 'good']

```

```

rows = []
for split in ["train", "test"]:
    for cls in ["good", "defective"]:
        folder = os.path.join(base_target, split, cls)
        for fname in os.listdir(folder):
            if fname.lower().endswith(IMAGE_EXTS):
                rows.append({
                    "filepath": f"milk_bottle_defects/{split}/{cls}/{fname}",
                    "split": split,
                    "class": cls,
                    "defect_type": "" if cls=="good" else "generic_defect"
                })

df_labels = pd.DataFrame(rows)
labels_path = "/content/milk_bottle_defects/labels.csv"
os.makedirs("/content/milk_bottle_defects", exist_ok=True)
df_labels.to_csv(labels_path, index=False)
print("labels.csv saved at:", labels_path)
print(df_labels.head())

```

```
labels.csv saved at: /content/milk_bottle_defects/labels.csv
```

	filepath	split	class	defect_type
0	milk_bottle_defects/train/good/images483_423_1...	train	good	
1	milk_bottle_defects/train/good/6138fH71kaL._SL...	train	good	
2	milk_bottle_defects/train/good/IMG202212101524...	train	good	
3	milk_bottle_defects/train/good/images435_398_1...	train	good	
4	milk_bottle_defects/train/good/CERAMIC-STOPPER...	train	good	

```
data_root = base_target
train_dir = os.path.join(data_root, "train")
test_dir = os.path.join(data_root, "test")

img_size = (128, 128)
batch_size = 16

train_ds = keras.utils.image_dataset_from_directory(
    train_dir, image_size=img_size, batch_size=batch_size,
    label_mode="categorical", shuffle=True, seed=42
)
test_ds = keras.utils.image_dataset_from_directory(
    test_dir, image_size=img_size, batch_size=batch_size,
    label_mode="categorical", shuffle=False
)

class_names = train_ds.class_names
print("Classes:", class_names)

AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
test_ds = test_ds.prefetch(AUTOTUNE)
```

```
Found 3587 files belonging to 2 classes.
Found 898 files belonging to 2 classes.
Classes: ['defective', 'good']
```

```
model = keras.Sequential([
    layers.Rescaling(1./255, input_shape=(128,128,3)),
    layers.Conv2D(16, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(2, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

history = model.fit(
    train_ds,
    epochs=5, # 5 is enough; your GPU time will be lower
    validation_data=test_ds,
    verbose=1
)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 128, 128, 3)	0
conv2d (Conv2D)	(None, 126, 126, 16)	448
max_pooling2d (MaxPooling2D)	(None, 63, 63, 16)	0
conv2d_1 (Conv2D)	(None, 61, 61, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 32)	0
conv2d_2 (Conv2D)	(None, 28, 28, 64)	18,496
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 64)	0
flatten (Flatten)	(None, 12544)	0
dense (Dense)	(None, 64)	802,880
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 2)	130

Total params: 826,594 (3.15 MB)

Trainable params: 826,594 (3.15 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/5

225/225 ————— 203s 892ms/step - accuracy: 0.8767 - loss: 0.3917 - val_accuracy: 0.8909 - val_loss: 0.3332

Epoch 2/5

225/225 ————— 156s 694ms/step - accuracy: 0.8883 - loss: 0.3129 - val_accuracy: 0.8909 - val_loss: 0.3163

Epoch 3/5

225/225 ————— 160s 714ms/step - accuracy: 0.8873 - loss: 0.2791 - val_accuracy: 0.8909 - val_loss: 0.3289

Epoch 4/5

225/225 ————— 158s 702ms/step - accuracy: 0.8896 - loss: 0.2605 - val_accuracy: 0.8909 - val_loss: 0.3681

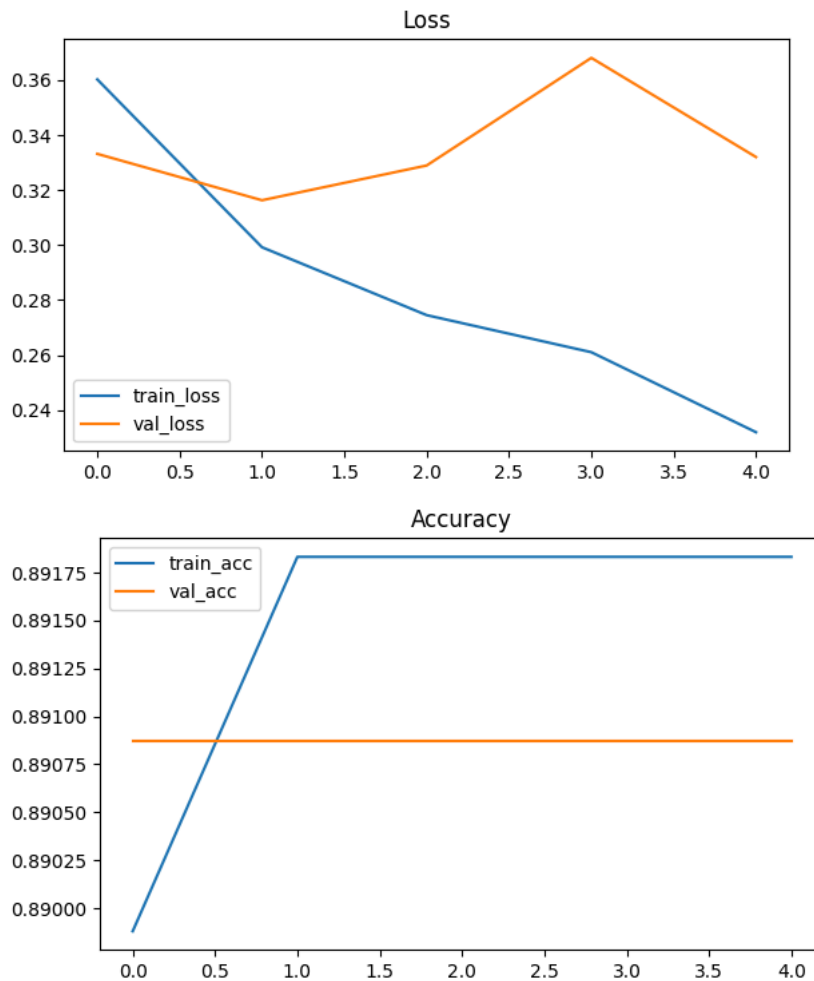
Epoch 5/5

```

plt.figure()
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.legend()
plt.title("Loss")
plt.show()

plt.figure()
plt.plot(history.history['accuracy'], label='train_acc')
plt.plot(history.history['val_accuracy'], label='val_acc')
plt.legend()
plt.title("Accuracy")
plt.show()

```

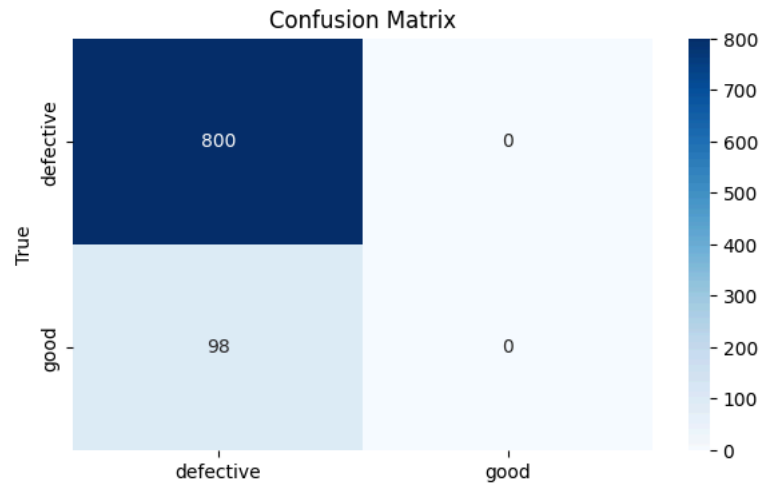


```
# Collect true & predicted labels
y_true = []
for _, labels in test_ds:
    y_true.extend(np.argmax(labels.numpy(), axis=1))
y_true = np.array(y_true)

y_pred = []
for imgs, _ in test_ds:
    preds = model.predict(imgs, verbose=0)
    y_pred.extend(np.argmax(preds, axis=1))
y_pred = np.array(y_pred)

cm = confusion_matrix(y_true, y_pred, labels=[0,1]) # ensure 2 classes order
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=class_names, yticklabels=class_names
)
plt.title("Confusion Matrix")
plt.ylabel("True")
plt.xlabel("Predicted")
plt.show()

print(classification_report(y_true, y_pred, target_names=class_names))
```



	precision	recall	f1-score	support
defective	0.89	1.00	0.94	800
good	0.00	0.00	0.00	98
accuracy			0.89	898
macro avg	0.45	0.50	0.47	898
weighted avg	0.79	0.89	0.84	898

```
plt.figure(figsize=(10,6))
n_show = 8

for imgs, labels in test_ds.take(1):
    preds = model.predict(imgs, verbose=0)
    for i in range(n_show):
        ax = plt.subplot(2,4,i+1)
        plt.imshow(imgs[i].numpy().astype("uint8"))
        p = class_names[np.argmax(preds[i])]
        t = class_names[np.argmax(labels[i].numpy())]
        plt.title(f"P:{p} / T:{t}")
        plt.axis("off")

plt.tight_layout()
plt.show()
```

```
labels_csv = "/content/milk_bottle_defects/labels.csv"

if os.path.exists(labels_csv):
    df = pd.read_csv(labels_csv)
    df['defect_type'] = df['defect_type'].fillna("")

    freq = df[df['class']=="defective"]['defect_type'].value_counts()
    print(freq)

    # Bar chart
    freq.plot(kind='bar', color='skyblue')
    plt.title("Defect Type Frequency")
    plt.xlabel("Defect Type")
    plt.ylabel("Count")
    plt.xticks(rotation=30, ha='right')
    plt.tight_layout()
    plt.show()

    # Word cloud
    text = " ".join(df[df['class']=="defective"]['defect_type'].tolist()) or "no_defects"
    wc = WordCloud(width=800, height=400, background_color="white").generate(text)
    plt.imshow(wc, interpolation='bilinear')
    plt.axis("off")
    plt.title("Defect Word Cloud")
    plt.show()
else:
    print(" labels.csv not found – skipping NLP add-on.")
```

```
defect_type
generic_defect    3999
Name: count, dtype: int64
```

